

บทที่ 8

ภาษาจาวา

(JAVA)

จาวาเป็นโปรแกรมขั้นสูง ที่พัฒนาขึ้นมาโดยบริษัท ซันไมโครซิสเต็มส์ (Sun Microsystems) เริ่มแรก Sun ได้คิดค้นภาษาจาวาขึ้นมาเพื่อใช้กับระบบปฏิบัติการ Set Top Box คือ โครงการของระบบตลาดอิเล็กทรอนิกส์ ที่ต้องการให้ระบบสามารถทำการควบคุมอุปกรณ์อิเล็กทรอนิกส์ได้โดยไม่ขึ้นกับแพลตฟอร์ม (Platform Independent) ซึ่งหมายความว่าพัฒนาระบบปฏิบัติการหรือซอฟต์แวร์ให้สามารถทำงานได้ โดยไม่สนใจว่าไมโครโปรเซสเซอร์จะใช้รุ่นใด ยี่ห้อไหน ฮาร์ดแวร์เป็นของอะไร ทั้งยังได้รวมความสามารถไปถึงการเชื่อมต่อ การแลกเปลี่ยนข้อมูลข่าวสารทั่วไปและทางด้านธุรกิจผ่านทางระบบเครือข่ายคอมพิวเตอร์

จาวาเป็นหนึ่งในภาษาที่ใช้ในการเขียนโปรแกรมเชิงวัตถุที่ใช้งานกันอย่างแพร่หลายในปัจจุบัน ซึ่งทางทีมงานถึงความเจริญเติบโตของภาษาจาวาและคุณลักษณะเด่นในด้านการพัฒนาโปรแกรมเชิงวัตถุ จึงเลือกภาษาจาวาเป็นภาษาในการพัฒนาโปรแกรมในโครงการ ในบทนี้จะกล่าวถึงลักษณะทั่วไปของภาษาจาวา

8.1 ความสามารถของภาษาจาวา

ความสามารถของภาษาจาวาสรุปได้ดังต่อไปนี้

- เป็นภาษาพัฒนาโปรแกรมเชิงวัตถุ
- การใช้งานที่ไม่ขึ้นกับแพลตฟอร์ม โดยใช้วิธีการแปลงข้อมูลแบบไบต์โค้ด (Byte code)
- การใช้เมธอดจัดการกับข้อมูลแทนการใช้ฟังก์ชัน
- ความสามารถในการดึงคลาสดาวน์โหลดจากที่ต่างๆ ผ่านทาง HTTP และ FTP
- ความสามารถในการโต้ตอบแบบเรียลไทม์
- เป็นระบบมัลติเธรด (Multithread) คือ สามารถทำงานได้หลายงานพร้อมกันในเวลาเดียวกัน
- ไม่มีตัวแปรแบบพอยเตอร์ จึงหมดปัญหาในการใช้งานหน่วยความจำ
- มีความปลอดภัยสูง

8.2 การนำจาวาไปประยุกต์สร้างและพัฒนาในงานด้านต่างๆ

เราสามารถนำภาษาจาวาสร้างและพัฒนาแอปพลิเคชันด้านต่างๆซึ่งกล่าวโดยสังเขปดังนี้

จาวาสามารถเป็นเครื่องมือในการพัฒนาแอปพลิเคชันด้านไคลเอนต์ โดยเฉพาะอย่างยิ่งด้านกราฟฟิก และระบบติดต่อกับผู้ใช้ ปัจจุบันโปรแกรมที่เขียนขึ้นจากภาษาจาวาจะทำงานที่ฝั่งไคลเอนต์เป็นส่วนใหญ่ เช่น จาวาแอปพลิเคชัน (Java Application) เป็นต้น

ส่วนการสร้างโปรแกรมประยุกต์บนเซิร์ฟเวอร์ จำเป็นต้องได้รับการปรับปรุงการทำงานอยู่สามอย่าง คือ การเชื่อมต่อฐานข้อมูลโดยตรง การทำงานให้สอดคล้องกับอินพุตและเอาต์พุต และความเร็วของการรัน ต้องเพิ่มขึ้นให้เท่ากับโปรแกรมประยุกต์ทั่วไป

จาวาได้สร้างสภาพแวดล้อมเสมือน Java Virtual Machine (JVM) ซึ่งอนุญาตเฉพาะ จาวาแอปพลิเคชัน และจาวาแอปเพล็ต ซึ่งเป็นไบนารีโค้ดเท่านั้นที่สามารถรันได้โดยที่ไบนารีโค้ดจะปราศจากไวรัสหรือส่วนที่จะทำอันตรายต่อระบบ อันเนื่องจาก JVM จะจำกัดสิทธิ์การเข้าใช้ทรัพยากรของไบนารีโค้ด ในอนาคตการรักษาความปลอดภัยอีกระบบหนึ่งที่กำลังถูกนำมาปรับใช้ คือ “การเข้ารหัสข้อมูล”

8.3 หลักการทำงานของภาษาจาวา

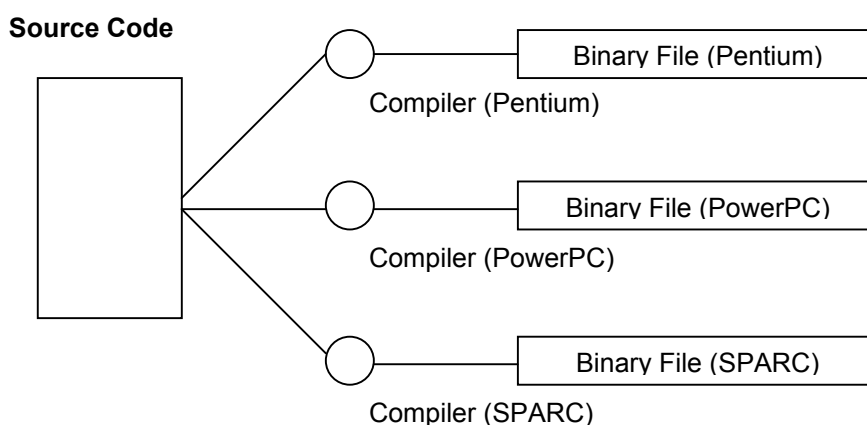
ในที่นี้จะอธิบายการทำงานของภาษาจาวาเริ่มต้นตั้งแต่การคอมไพล์ตัวโปรแกรมจนกระทั่งเราเรียกใช้งานแอปพลิเคชันได้

8.3.1 การคอมไพล์เลอร์

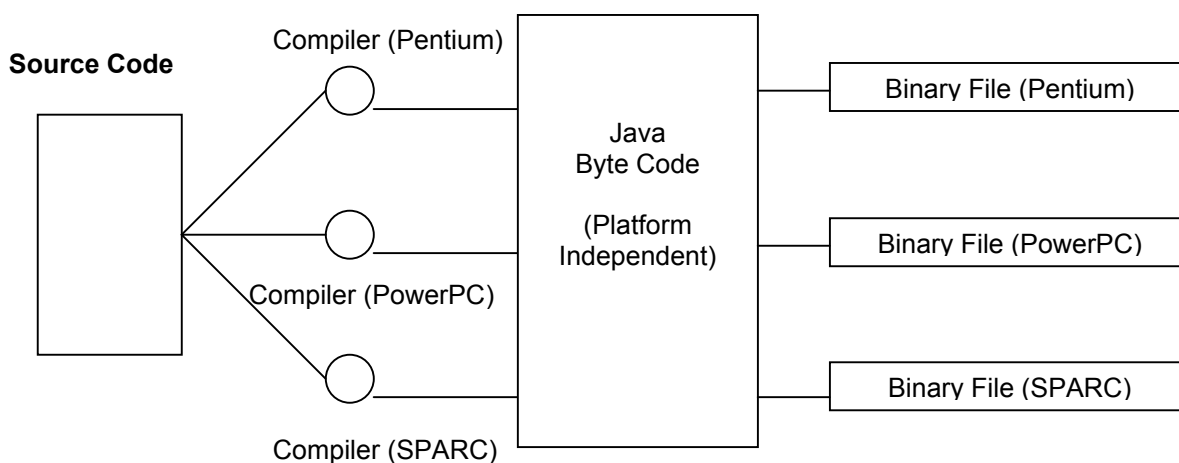
การคอมไพล์ของภาษาจาวาก็เช่นเดียวกับคอมไพล์เลอร์ภาษาอื่นๆ คือ มันจะสร้างรหัสภาษาเครื่อง (Machine Code หรือ Assemble Code) จากภาษาในระดับที่สูงกว่าเพื่อให้คอมพิวเตอร์ (Central Control Unit : CPU) สามารถแปลความหมายและนำไปประมวลผลต่อไปได้

แต่ข้อแตกต่างที่สำคัญระหว่างคอมไพล์เลอร์ของภาษาจาวากับภาษาอื่นๆ คือ CPU ที่จะทำหน้าที่ปฏิบัติตามคำสั่งที่ได้จากการคอมไพล์ภาษาจาวานั้นไม่มีอยู่จริง เป็นเพียงสิ่งที่สมมติขึ้นมาที่เรียกว่า Java Virtual Machine นอกจากนี้การอ้างถึงส่วนต่าง ๆ ของโปรแกรมที่คอมไพล์ด้วยคอมไพล์เลอร์ภาษาจาวาก็จะมีวิธีการที่แตกต่างออกไป

คอมไพล์เลอร์ภาษาจาวาจะไม่เปลี่ยนการอ้างถึงส่วนของโปรแกรมจากการใช้ชื่อแบบในภาษาระดับสูงไปเป็นตัวเลขเหมือนที่คอมไพล์เลอร์ภาษาอื่น ๆ ทำกัน และคอมไพล์เลอร์ของภาษาจาวาจะไม่มีการสร้างแผนที่ของการจัดวางโปรแกรมบนหน่วยความจำขึ้นมาในระหว่างการคอมไพล์ ด้วยเหตุผลที่สำคัญคือ เพื่อเป็นการสร้างพอร์ตเทเบิล (Portable) ให้กับตัวโปรแกรม การจัดวางตำแหน่งของโปรแกรมจะต้องขึ้นอยู่กับลักษณะการทำงานของโปรเซสเซอร์ตัวใดตัวหนึ่ง การยังไม่จัดวางตำแหน่งช่วยให้โปรแกรมที่ได้จากการคอมไพล์มีความเป็นกลาง สามารถนำไปใช้บนคอมพิวเตอร์แพลตฟอร์มอื่นๆได้ นอกจากนี้ยังทำให้เกิดความปลอดภัยอีกด้วย สิ่งที่ได้จากการคอมไพล์ภาษาจาวาเรียกว่า ไบนารีโค้ด (Byte Code)



รูปที่ 8.1 การประมวลผลของโปรแกรมทั่วไปแต่ระบบปฏิบัติการที่ต่างกัน



รูปที่ 8.2 การประมวลผลของโปรแกรมภาษาจาวา

8.3.2 การวางตำแหน่งในหน่วยความจำ

ภาษาจาวาไม่มีการลดรูปแบบการอ้างอิงถึงส่วนต่างๆ ของโปรแกรมจากการเรียกเป็นชื่อให้เหลือเพียงตัวเลขหรือแอดเดรสที่กำหนดขึ้นจากการจัดวางตำแหน่งของโปรแกรมลงในหน่วยความจำ คอมไพเลอร์ภาษาจาวาจะทิ้งชื่อของแต่ละส่วนของโปรแกรม เอาวางไว้ในตัวโปรแกรมที่สร้างขึ้น เมื่อโปรแกรมทำงานจะเป็นหน้าที่ของตัวอินเตอร์พรีเตอร์ (interpreter) ที่จะคอยเปิดตารางค้นหาที่อยู่ของเมธอดที่ต้องการเรียกใช้งาน โดยก่อนที่จะเริ่มทำงานจริง อินเตอร์พรีเตอร์ จะต้องสร้างแผนที่ในการจัดการวางสิ่งต่างๆ ลงในหน่วยความจำลงในหน่วยความจำขึ้นมาก่อนแล้วจึงสร้างตารางขึ้นมาใหม่ เพื่อช่วยหาดำแหน่งของเมธอดเมื่อมีการเรียกใช้งาน โดยใช้ชื่อของเมธอด

8.3.3 การรันโปรแกรม

การรันโค้ดที่คอมไพล์เอาไว้สำหรับ Java Virtual Machine เป็นหน้าที่ของตัวอินเตอร์พรีเตอร์การรันโปรแกรมจะแบ่งเป็น 3 ขั้นตอนหลัก ๆ คือ การอ่าน การตรวจสอบความถูกต้อง และการทำตามโค้ด หน้าที่ในการอ่านโค้ดเข้าสู่ระบบจะเป็นของ Class Loader หน้าที่การทำงานในส่วนนี้จะไม่ได้เข้ามาเฉพาะไฟล์จาวาที่กำลังจะเรียกใช้เท่านั้น แต่จะอ่านคลาสที่มีการอ้างอิง และคลาสที่มีการ inherited มาโดยคลาสที่อ้างอิง เมื่อผ่านขั้นตอนนี้แล้ว โค้ดทั้งหมดก็จะถูกส่งผ่านตัวตรวจสอบไบนารีโค้ดเพื่อให้แน่ใจว่าโค้ดที่ส่งมามีความถูกต้องตามมาตรฐานของจาวา และจะไม่รบกวนเสถียรภาพระบบ เมื่อผ่านการตรวจสอบแล้ว โค้ดก็จะถูกส่งต่อไปยังระบบรันไทม์ (Run-Time System) ซึ่งจะส่งงานไปยังฮาร์ดแวร์อีกต่อหนึ่ง หลักการทำงานในแต่ละขั้นตอนที่กล่าวมาข้างต้นเป็นดังนี้

8.3.4 คลาสโหลดเดอร์(Class Loader)

ตัวคลาสโหนดเดอร์ จะทำหน้าที่ดั่งโค้ดทั้งหมดที่จำเป็นในการทำงานของแอปพลิเคชัน ไม่ว่าจะเป็นคลาสที่ถูก Inherited มาหรือคลาสอื่น ๆ ที่มีการเรียกใช้ เมื่อโหนดเดอร์ดึงคลาสใดเข้าแล้วก็จะจัดคลาสนั้นๆ ได้เข้าไว้ใน namespace ของมันเอง โดยการเก็บจะใช้ชื่อคลาสเป็นสำคัญ ไม่ได้ใช้การอ้างถึงเป็นตัวเลข หลักการนี้ก็จะเหมือนกันกับการทำงานของ Virtual Machine ที่ระบบปฏิบัติการสร้างขึ้นให้กับแอปพลิเคชันแต่ละตัวทำงาน ถ้าไม่ได้มีการเจาะจงเรียกใช้คลาสที่มีอยู่นอก namespace นี้ การเรียกใช้ชื่อต่าง ๆ ในคลาสก็จะไม่มีการรบกวนกันระหว่างคลาสเลย

คลาสทั้งหมดที่อยู่บนเครื่องโลคอล (Local) เอง จะได้รับห้วงแอดเดรส (Address Space) เป็นของตัวเอง ส่วนคลาสต่าง ๆ ที่ดึงมาจากภายนอกจะได้รับ namespace เป็นของตัวเอง การทำงานลักษณะนี้จะช่วยให้คลาสที่อยู่บนโลคอลทำงานได้ประสิทธิภาพดีขึ้น เพราะใช้ namespace ร่วมกันได้ แต่ก็ยังมีทางป้องกันความผิดพลาดที่อาจเกิดจากคลาสที่ดึงเข้ามาจากคลาสภายนอก และในทางกลับกัน คลาสที่อิมพอร์ต (import) เข้ามา ก็ปลอดภัยจากความผิดพลาดที่อาจเกิดจากคลาสโลคอลด้วย

เมื่อคลาสทั้งหมดที่เกี่ยวข้องกับการทำงานถูกอิมพอร์ตเข้ามาเรียบร้อยแล้ว การจัดวางหน่วยความจำสำหรับเริ่มต้นการทำงานก็จะเกิดขึ้นได้ การเรียกชื่อต่างๆ ก็จะสามารถจับคู่กับแอดเดรสจริง ๆ ของหน่วยความจำได้ แล้วตัวโหนดเดอร์จะสร้างตารางสำหรับค้นหาที่อยู่เนื่องจากการทำงานผิดปกติของซูเปอร์คลาส และการอ้างอิงแอดเดรสที่ไม่ถูกต้องได้

8.3.5 ตรวจสอบไบท์โค้ด

เมื่อโค้ดเดินทางมาถึงขั้นตอนการสร้างตารางจับคู่ชื่อกับแอดเดรสแล้ว ก็ยังไม่สามารถแน่ใจได้ว่าโค้ดที่อ่านเข้ามาจะมีความปลอดภัย ดังนั้นจึงต้องมีตัว verifier หรือตัวตรวจสอบไบท์โค้ด ทำหน้าที่ตรวจสอบความถูกต้องที่ละบรรทัดว่าเป็นไปตามข้อกำหนดของจาวา และสอดคล้องกับการทำงานของตัวโปรแกรมเองหรือไม่ การตรวจสอบโค้ดในเชิงทฤษฎีจะสร้างค้นหาปัญหาต่างๆ ได้หลายอย่าง เช่น จะไม่มีการสร้างพอยเตอร์ที่เกินกว่าหน่วยความจำจริง ไม่มีคำสั่งใดสามารถละเมิดสิทธิ์การทำงานของตัวโปรแกรมได้ ไม่มีการจับคู่ ไม่มีการจับคู่ออบเจ็กต์ผิด จะไม่มีการให้ operand มากหรือน้อยเกินไป การกำหนดค่าต่างๆ สำหรับไบท์โค้ดจะต้องถูกต้องครบถ้วน และจะไม่มีการแปลงข้อมูลผิดรูปแบบ

การใช้ตัวตรวจสอบตอบสนองจุดประสงค์ 2 ประการสำคัญ คือสิ่งต่าง ๆ ดังกล่าวมาแล้วจะถูกตรวจสอบก่อนทำให้ตัวอินเตอร์พรีเตอร์มั่นใจได้ว่า ไบท์โค้ดที่ส่งเข้าไปทำงานจะไม่มีขั้นตอนการทำงานที่สร้างปัญหาให้กับตัวระบบ และจุดประสงค์ที่สองคือ ตัวอินเตอร์พรีเตอร์จะทำงานตามไบท์โค้ดได้เร็วกว่า เพราะไม่ต้องคอยระวังว่าจะมีปัญหาเกิดขึ้น และไม่ต้องหยุดเป็นช่วงๆ เมื่อพบปัญหาและต้องแก้ไข

ในการทำงาน ไบท์โค้ดจะถูกตรวจสอบเพียงครั้งเดียวเท่านั้น และจะทำงานไปได้ตลอดไม่ต้องมีการตรวจสอบซ้ำอีกเมื่อมีการเรียกกลับมาทำงานที่ส่วนเดิมของโปรแกรม

8.3.6 การทำงานตามโค้ด

เมื่อตัวโหลดเคอร์ได้รวบรวมโค้ดเข้ามาสู่ระบบ ระบบก็จะจัดวางในหน่วยความจำ และตัวตรวจสอบได้ตรวจสอบความถูกต้องแล้ว โค้ดก็จะถูกส่งต่อไปยังตัวอินเทอร์พรีเตอร์ทำงานตามคำสั่ง การทำงานตามคำสั่งของไบท์โค้ดก็คือ การเปลี่ยนโค้ดให้กลายเป็นคำสั่งการทำงานจริงที่ตัวระบบไคลเอนต์ที่รันโค้ดนี้สามารถทำงานได้ ซึ่งวิธีการที่ทำได้มีอยู่ 2 วิธีด้วยกันคือ ตัวอินเทอร์พรีเตอร์ทำการคอมไพล์โค้ดเหล่านี้ให้กลายเป็นเนทีฟโค้ด (native code) ที่ตัวไคลเอนต์เข้าใจได้ แล้วค่อยทำงาน เพื่อให้ได้ความเร็วสูงสุดในการทำงาน กับอีกวิธีหนึ่งก็คือ อินเทอร์พรีเตอร์อ่านโค้ดมาแล้วตีความหมายการทำงานไปที่ละคำสั่ง และตีความไปเรื่อยตลอดการทำงาน

โดยปกติแล้ว ผู้สร้างอินเทอร์พรีเตอร์มักจะเลือกใช้วิธีการหลัง รูปแบบของไบท์โค้ดในภาษาจาวามีความยืดหยุ่นเพียงพอที่จะสามารถเปลี่ยนไปทำงานบนเครื่องไคลเอนต์แบบต่างๆ ได้โดยไม่มีการก่อให้เกิดโอเวอร์เฮด (overhead) มากเกินความจำเป็น อย่างไรก็ตามไคลเอนต์ของจาวาบางระบบมีความสามารถในการทำงานได้ทั้งสองวิธี คือ โปรแกรมเมอร์สามารถจะเลือกใช้วิธีการคอมไพล์กับงานที่เน้นการคำนวณมากๆ เพื่อเป็นการเพื่อสมรรถนะในการทำงานให้ได้เต็มที่

การสร้างระบบรันไทม์ที่ดีจะต้องถ่วงดุลความสำคัญ 3 ประการให้ได้พอเหมาะ นั่นคือ portable, ความปลอดภัย และสมรรถนะ ในเรื่องของ portable ทำได้โดยการใช้รูปแบบของไบท์โค้ดที่มีความเป็นกลางเพียงพอ สามารถนำไปใช้รันบนเครื่องคอมพิวเตอร์เครื่องต่างๆ ได้โดยง่าย นอกจากนั้น การที่ตัวอินเทอร์พรีเตอร์กำหนดการจัดการตำแหน่งหน่วยความจำในช่วงรันไทม์ แทนที่จะเป็นระหว่างคอมไพล์เหมือนภาษาอื่นๆ ก็เป็นการเพิ่มความแน่นอนว่า คลาสต่างๆ ที่ อิมพอร์ตเข้ามาจะยังคงใช้ได้อยู่ตลอดเวลา ส่วนเรื่องความปลอดภัยเป็นสิ่งที่มีการคำนึงถึงตลอดกระบวนการรันไทม์ โดยเฉพาะอย่างยิ่งตัวตรวจสอบไบท์โค้ด ที่ทำให้แน่ใจว่า โปรแกรมจะทำงานได้ถูกต้องตามข้อกำหนดของจาวา ส่วนเรื่องของสมรรถนะนั้นก็สามารถจัดการได้ในสองระยะ คือ พยายามเอาโอเวอร์เฮดทั้งหลายไปใส่ไว้ที่ตอนเริ่มต้นโหลดโปรแกรมเข้ามาสู่ระบบ หรือไม่ก็กำหนดให้ทำงานเป็นแบบแบ็กกราวนด์ (Back-Ground Thread)

ด้วยตัวอย่างเหล่านี้ ทำให้จาวาสามารถแสดงสมรรถนะในระดับที่น่าพอใจออกมาได้โดยที่ยังคงไว้ซึ่งพอร์ตเทเบิล และสภาพแวดล้อมที่ปลอดภัย นอกจากนั้น ยังสามารถจะดึงสมรรถนะระดับสูงสุดออกมาใช้ได้ทันทีเมื่อต้องการ

8.4 การสร้างและรันภาษาจาวา

การสร้างโปรแกรมด้วยภาษาจาวาก็เหมือนกับภาษาอื่นๆ คือเขียนโปรแกรมจาก Editor ใดๆ ก็ได้ บันทึกอยู่ในนามสกุล .java จากนั้นจึงนำไปคอมไพล์ด้วยคอมไพเลอร์ของ Java จะได้ไฟล์ใหม่นามสกุล .class ที่เก็บรหัสของการคอมไพล์ไว้ จาวาเรียกรูปแบบข้อมูลที่อยู่ในไฟล์ใหม่นี้ว่า Byte Code ไฟล์ที่ได้จะไม่สามารถรันได้ทันทีเหมือนไฟล์นามสกุล .exe หรือ .com ที่เรารู้จักกัน เพราะข้อมูลแบบ Byte Code จะมีรูปแบบข้อมูลที่อยู่กึ่งกลางระหว่างโค้ดโปรแกรม (Source Code) กับโค้ดที่คอมพิวเตอร์อ่านแล้วนำไปปฏิบัติงานได้ทันที (Machine Code) หากจะรันไฟล์ .class บนระบบใดๆ ต้องใช้อินเทอร์พรีเตอร์จาวาของระบบนั้นๆ

เพื่อตรวจสอบความถูกต้องของ Byte Code แล้วแปลให้เป็นรหัสภาษาเครื่องส่งให้ระบบปฏิบัติการทำไปรันต่อไปขั้นตอนการทำงาน

ลักษณะของการเขียน โปรแกรมภาษาจาวาเป็นการเขียน โปรแกรมเชิงวัตถุ (Object Oriented Programming : OOP) ตัวโปรแกรมต้องถูกคอมไพล์เป็นโค้ด ซึ่งเรียกว่า Java Byte Code เครื่องมือในการพัฒนาภาษาโปรแกรมจาวามีการจำลองตัวมันเองเป็น JVM (Java Virtual Machine) ซึ่งเป็นการสมมติให้มีคอมพิวเตอร์อีกเครื่องหนึ่งขึ้นมาโดยเครื่องนี้จะใช้สำหรับคอมไพล์ภาษาจาวาทุกโปรแกรม และเอ็กคิวต์ (execute) คำสั่งแต่ละในจาวาโค้ดให้คอมพิวเตอร์ทำงานตามคำสั่งได้ ข้อดีของการเขียน โปรแกรมจาวามีดังนี้

- เป็นภาษาที่เรียนรู้ได้อย่างรวดเร็ว
- ในการเขียนโปรแกรมหนึ่งๆ ขนาดของโค้ดจะเล็ก
- สามารถนำไปใช้งานกับเครื่องประเภทอื่นได้

การเขียนโปรแกรมภาษาจาวาที่เป็นที่นิยมมี 2 รูปแบบคือ

- จาวาแอปพลิเคชัน เป็นโปรแกรมแบบ stand alone ไม่สามารถใช้งานบนเว็บเบราว์เซอร์ได้
- จาวาแอปเพลต สามารถใช้งานบนเว็บเบราว์เซอร์ได้

การเขียนโปรแกรมภาษาจาวานั้น เราต้องติดตั้งตัวแปลภาษาจาวาก่อน ซึ่งในการพัฒนาโครงการนี้ เราได้เลือก JDK 1.4 เป็นคอมไพเลอร์ เราสามารถใช้คำสั่ง javac เพื่อคอมไพล์โปรแกรม ผลที่ได้จากการคอมไพล์โปรแกรมจาวา คือ ไฟล์นามสกุล .class เป็นเหมือนกับไฟล์เอ็กคิวต์ของจาวา การเรียกให้ไฟล์ .class นี้ทำงานไม่ต้องใส่นามสกุล .class

การเขียนโปรแกรมภาษาจาวาจะต้องกำหนดให้ชื่อไฟล์กับชื่อเดียวกัน เพราะในการคอมไพล์ หรือ รันโปรแกรม เราจะเรียกโดยใช้ชื่อนั้นเอง ซึ่งไฟล์ .class ของจาวาสามารถทำงานข้ามแพลตฟอร์มได้ และตรงนี้เองทำให้ภาษาจาวาได้รับความนิยมและนำไปใช้งานมากขึ้น

ตัวอย่าง การใช้จาวา แสดงข้อความ Hello World ! ออกมาที่คอมมานด์ไลน์ (command line)

1. เขียนซอร์สโค้ดชื่อ Hello.java โดยมีรายละเอียดโค้ดดังนี้

```
Class Hello {
    public static void main(String arg[]) {
        System.out.println("Hello World !");
    }
}
```

2. คอมไพล์ซอร์สโค้ด โดยใช้คำสั่ง `# javac Hello.java`
ซึ่งในขั้นตอนนี้จะได้ไฟล์ Hello.class

3. รัน Hello.class ด้วยคำสั่ง `#java Hello`

ผลที่ได้จากการรัน Hello.class เป็นดังนี้

```
#java Hello
```

```
Hello World !
```